Object_Detection

2D

1. Indirect methods (two-stage): Faster R-CNN
2. Direct methods (one-stage):
    1. anchor based: YOLO, RetinaNet
    2. anchor-free/one anchor: , CenterNet, FCOS
3. Faster R-CNN
    1. ROI pooling is tf.image.crop_and_resize
4. YOLO
    1. YOLO treats object detection as a **regression problem**. Specifically, we encode the gt bbox centers as offsets from the top-left corner of each cell in the feature map into 0-1 range (as we've known neural networks like to work with values in the range -1 to 1 or 0 to 1). The network will then learn to regress these offsets which is in the range 0-1.
    2. The image is resized to (image_size, image_size) before input to the backbone. For example image_size=224 or 448 are commonly used (224 or 448 for Resnet backbone, that outputs 7x7 or 14x14 feature map).
    3. The TLWH box format from COCO is converted to $x_1y_1x_2y_2$ format and divided by (w,h,w,h).
    4. The box encoder purpose is to provide (image, target) pair for training. In box encoder the boxes in $x_1y_1x_2y_2$ format is converted to x_cy_cwh format. The offset from the center of each bbox to the top-left corner of each grid cell which is $delta\_xy$ is computed by :
        1. For instance, if S=num_cell=7 (or 14), then cell_size = 1/7.
        2. To convert to grid cell coordinate divide x_cy_c by cell_size. To convert back to image coordinate multiply by $cell\_size$.
        3. In grid cell coordinate, $ij = floor(cxcy\_sample/cell\_size)$, top-left corner (in image coordinate): $xy = ij * cell\_size$. The offset in grid cell coordinate is $delta\_xy = (cxcy\_sample - xy)/cell\_size$. Now, $delta\_xy$ is in the range [0,1] in grid cell coordinate and is to be regressed by the network.
    5. YOLO outputs a feature map tensor of size $(S, S, B * 5 + C)$. B is the number of candidate boxes. Note: there are no anchor boxes in YOLOv1.
    6. why 5? because the box is of the format $[\delta_x, \delta_y, w, h, conf]$ plus C possible classes for the one box predicted per grid cell.
    7. In YOLOv1 it is assumed that there is only one true box per grid cell. Therefore, to match the size of the output tensor (for computing loss). In the third axis of size $B * 5 + C$ only one ground truth box is used and is repeated B consecutive times throughout the third axis.
    8. To recover $x_1y_1x_2y_2$ format to compute IOU, because width and height of box is always in image coordinate it does not need to be multiplied by $cell\_size$:
        1. $box\_xyxy[:, :2] = box1[:, :2] * cell\_size - 0.5 * box1[:, 2:4]$
        2. $box\_xyxy[:, 2:4] = box1[:, :2] * cell\_size + 0.5 * box1[:, 2:4]$
        3. This is done so because when combining the two information they should share the same coordinate scale.
        4. Note: normally a function to compute IOU expects $x_1y_1x_2y_2$ format.
    9. Box width and height are regressed directly as after normalization by image_size they are in the range[0,1] in image coordinates.
    10. During training, bounding box assignment is done to assign one candidate ground truth box (since there are B copies of the same ground truth box) to B prediction boxes in each grid cell based on their IOUs. Note: not all B predicted boxes may ever get to be assigned during training. Note: This part is replaced with Hungarian algorithm in DETR, which is used during training only.
    11. The last activation function in the model is a Sigmoid function.
    12. The loss function is in the form:
    13. $L = L_{Localisation} + \lambda_{coobj} * L_{coobj} + \lambda_{noobj} * (L_{noobj} + L_{coo\_no\_response}) + L_{classification}$
    14. What I learned from implementing YOLOv1 from scratch:
        1. Training to improve mAP is tough. After 100 epochs the model only achieved ~2% mAP on the CODA dataset.
        2. The idea of using anchor boxes becomes obvious.
        3. Ways to improve mAP include hyperparameter tuning such as grid search using wandb sweeps.
5. CenterNet
    1. Objects as points.

2. Starting with image $I \in \mathbb{R}^{W \times H \times 3}$
3. Training for keypoint classification:
    1. The network will produce a keypoint heatmap $\hat{Y} \in [0,1]^{\frac{W}{R} \times \frac{H}{R} \times C}$
        1. R is the output stride
        2. C is the no. of keypoint types, for example:
            1. C=17 in human pose estimation
            2. C=80 object categories in object detection (COCO)
        3. use R=4, default in literature. R is downsampling stride. Image is downsampled R times in resolution.
    2. To construct the target $Y \in [0,1]^{\frac{W}{R} \times \frac{H}{R} \times C}$,
        1. Let the ground truth keypoint in image pixel coordinate be $p = (p_x, p_y)$. Then compute $\tilde{p}$ the lower resolution equivalent of p as:
            1. For example image is of size 28x28x3 divided by 4x4 tiles to become 7x7 grid cell. Say p=(26,27), R=4,
                1. $\tilde{p} = \frac{P}{R} = (\frac{26}{4}, \frac{27}{4}) = (6.5, 6.75)$, $\lfloor \frac{P}{R} \rfloor = (6, 6)$,
                2. $\tilde{p} = \frac{P}{R} = (\frac{25}{4}, \frac{24}{4}) = (6.25, 6.)$, $\lfloor \frac{P}{R} \rfloor = (6, 6)$.
        2. (x, y) being the coordinate of Y.
        3. Imagine point $(\tilde{p}_x, \tilde{p}_y)$ in the center and 8 points (x_i, y_i) surrounding it.
        4. Then for each class c, splat the cth dimension of Y with:
        5. $Y_{xyc} = \exp\left(-\frac{(x-\tilde{p}_x)^2 + (y-\tilde{p}_y)^2}{2\sigma_p}\right)$.
        6. If two gaussians of the *same* class overlap, we take the element wise maximum.
        7. The training objective is pixel wise logistic regression with focal loss:
        8. $$L_k = \sum_{xyc} -\frac{1}{N} \begin{cases} (1 - \hat{Y}_{xyc})^{\alpha} \log(\hat{Y}_{xyc}) & \text{if } Y_{xyc} = 1 \\ (\hat{Y}_{xyc})^{\alpha} \log(1 - \hat{Y}_{xyc})(1 - Y_{xyc})^{\beta} & \text{, otherwise} \end{cases}$$
4. Training for keypoint localisation:
    1. $$L_{off} = \text{localisation error in x coordinate} + \text{localisation error in y coordinate}$$
    2. $$L_{off} = \frac{1}{N}\left(\sum_{\tilde{p}_x} |\hat{O}_{\tilde{p}_x} - (\frac{p_x}{R} - \lfloor \frac{p_x}{R} \rfloor)| + \sum_{\tilde{p}_y} |\hat{O}_{\tilde{p}_y} - (\frac{p_y}{R} - \lfloor \frac{p_y}{R} \rfloor)|\right)$$
5. Training to recover width and height:
    1. kth Keypoint: $p_k = (p_x, p_y)$ with bbox:(x_1, y_1, x_2, y_2), $p_k = (\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2})$
    2. $\hat{S}_k$: predicted bbox width &height, $(\hat{x}_2^{(k)} - \hat{x}_1^{(k)}, \hat{y}_2^{(k)} - \hat{y}_1^{(k)})$
    3. S_k : ground truth width & height, $(x_2 - x_1, y_2 - y_1)$
    4. Without normalizing the scale and directly use raw pixel coordinates.
    5. $$L_{size} = \frac{1}{N} \sum_{p_k} |\hat{S}_{p_k} - S_k|$$
6. Then the overall training loss:
    1. $$L = L_k + \lambda_{size} L_{size} + \lambda_{off} L_{off}$$
    2. we set $lambda_{size} = 0.1$ and $\lambda_{off} = 1$, means penalize object center offset more than fitting object width & height.
7. The final output size is $(\frac{W}{R}, \frac{H}{R}, C + 4)$

6. DETR
    1. Compared to YOLO approach:
        1. Object query in place of anchor boxes.
        2. During training Hungarian matching in place of 'NMS' like bounding box assignment.
        3. As a result DETR is NMS free (no need of NMS during training or inference).
    2. DETR predicts in parallel the final set of detections.
    3. The CNN is used for feature learning, the transformer is used to make predictions.
    4. Object detection as set prediction.
    5. Using transformers and parallel decoding.
    6. Eliminates duplicate predictions.
    7. multiclass: dog: 0.9 -- uses Softmax activation function or cross entropy loss.

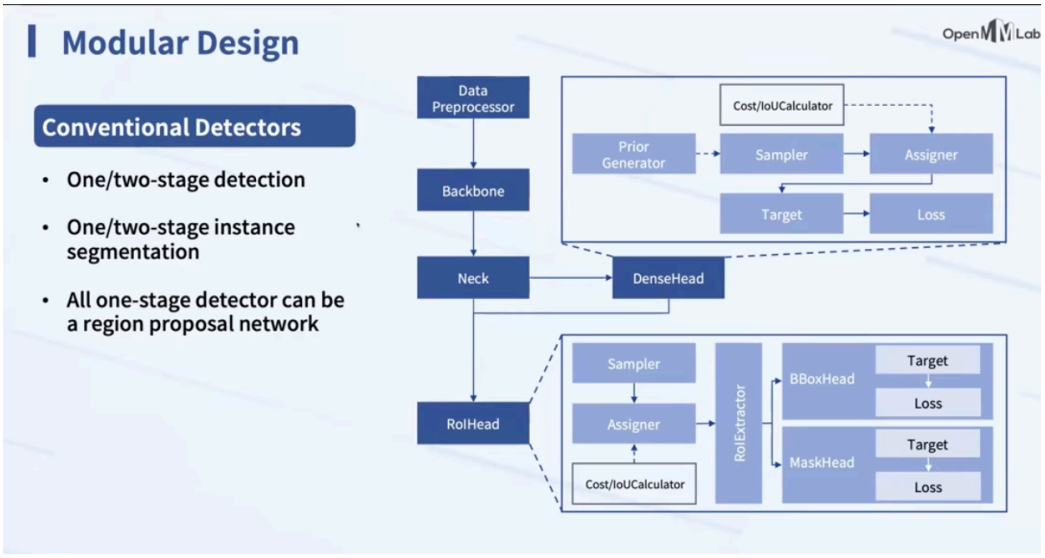
8. multilabel: dog: 0.9, trees: 0.75, -- uses Sigmoid or binary cross entropy loss, set prediction.
9. Code: dummy example of multi label classification in pytorch.

```
import torch
import torch.nn as nn
import torch.optim as optim

model = nn.Linear(20, 5) # predict logits for 5 classes
x = torch.randn(1, 20)  # batch_size = 1
y = torch.tensor([[1., 0., 1., 0., 0.]]) # get classA and classC as active

criterion = nn.BCEWithLogitsLoss()
optimizer = optim.SGD(model.parameters(), lr=1e-1)

for epoch in range(20):
    optimizer.zero_grad()
    output = model(x)
    print('output.shape', output.shape)  # (batch_size, n_classes): (1, 5)
    loss = criterion(output, y)
    loss.backward()
    optimizer.step()
    print('Loss: {:.3f}'.format(loss.item()))
```

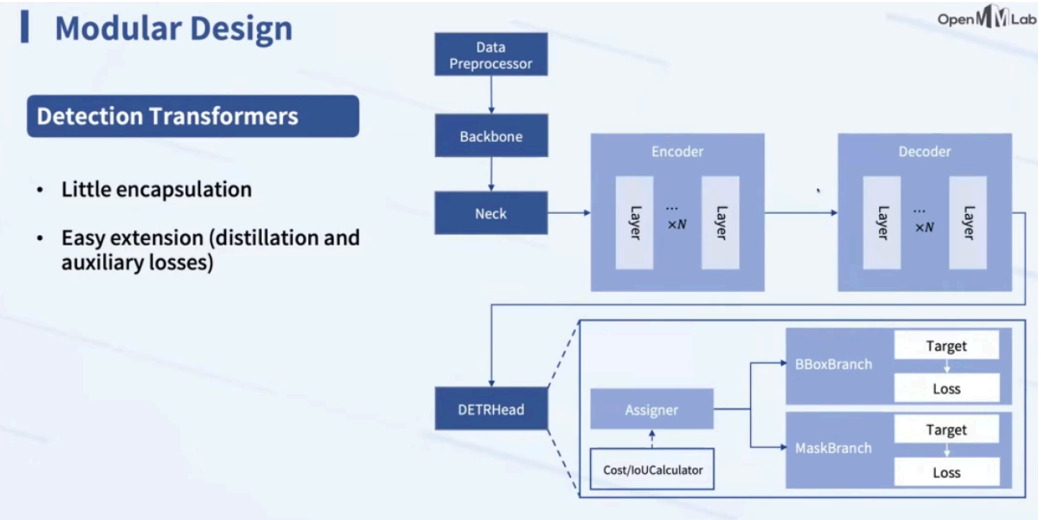
10. Issue with multilabel classification is when the model predicts the same amount of probability for two different objects. In object detection, if the region proposals are too near to each other.
11. In the context of object detection, DETR predicts N number of object boxes and N is much larger than the amount of ground truth bounding boxes G.
12. Bipartite matching is used to match the set of detections $\mathbb{N}$ to set of ground truth boxes in $\mathbb{G}$. Note: Bipartite matching is the process of pairing vertices from two sets in such a way that each vertex is paired with at most one vertex from the other set, and the total number of paired vertices is maximized.

7. SPP
    1. Used to adaptively produce feature maps of constant sizes, by recomputing the necessary kernel sizes and strides.
8. FPN
    1. Used to perform multi-scale object prediction.

    mmdetection2d :One-stage detectors can be viewed as a Region Proposal Network in a two-stage network, which is the upper box in this image.



DETR



3D

References:

1. Multi-label classification